

Ansible Inventory:

- An inventory defines a collection of host that ansible will manage.
- In inventory you can list managed hosts as group for easier collective management.
- Groups can contain child groups and host can be member of multiple groups.
- Host inventories can be define in two ways:
 - Static Invenevtory: defined in text file.
 - Dynamic inventory: can be generated by script or other program as need, using external information providor.

How to specify managed hosts in static inventory?

You can write in number of different formats:

- INI style ---> most common
- YAML file

Example:

web1.example.local

192.168.244.133

✓ For easier collective management, you can mention hosts in groups:

[webserver]

web1.example.local

192.68.100.10

web2-exaple.local

[database]

db1.example.local

db2.example.local

✓ You can also define nested groups , that are --> group of host groups:

Example:

[usa]

washington1.example.com

washington2.example.com

[canada]

ontario01.example.com

ontario02.example.com

[north-america:children]

canada

✓ A group can have both managed hosts and child group as members.

[north-america:children]

Canada

Dc1.lab.com

✓ You can specify ranges also:

[start:end]

Example:

192.168.[2:7].10

this includes- hosts from 192.168.2.10 192.168.3.10 until 192.168.7.10

web[a:f].example.local -> includes--> weba.example.local untill webf.example.local

verify the inventory:

✓ use Ansible command to verify the machine's presence in inventory:

[user@controlnode ~]\$ ansible washington1.example.com --list-hosts

✓ You can run the following command to list all hosts in a group:

[user@controlnode ~]\$ ansible canada --list-hosts

The “all” host group contains every host explicitly listed in the inventory. The “ungrouped” host group contains every host explicitly listed in the inventory that is not a member of any other group.

IMPORTANT If the inventory contains a host and a host group with the same name, the ansible command prints a warning and targets the host. The host group is ignored. There are various ways to deal with this situation, the easiest being to ensure that host groups do not use the same names as hosts in the inventory.

Location of inventory file:

- ✓ default location of static inventory file is :

`/etc/ansible/hosts`

- ✓ you can define in the current working directory, but you have to use -i inventory switch:

```
[student@workstation deploy-inventory]$ ansible testing -i inventory --list-hosts
```

Ansible Configuration file:

- ✓ The ansible package provides a base configuration file located at `/etc/ansible/ansible.cfg`.
- ✓ This file is used if no other configuration file is found.

Ansible configuration file precedence: Ansible will use configuration file in following precedence.

- 1: location specified in `ANSIBLE_CONFIG` environment variable.
- 2: Current working directory.
- 3: user home directory. `~/.ansible.cfg`
- 4: default location `/etc/ansible/ansible.cfg`

Note:

- ✓ The recommended practice is to create `an ansible.cfg` file in a directory from which you run Ansible commands.

- ✓ This directory would also contain any files used by your Ansible project, such as an inventory and a playbook.
- ✓ This is the most common location used for the Ansible configuration file.
- ✓ It is unusual to use a `~/ansible.cfg` or `/etc/ansible/ansible.cfg` file in practice.
- ✓ You can run the `ansible --version` command to clearly identify which version of Ansible is installed, and which configuration file is being used.

```
[user@controlnode ~]$ ansible --version
```

OR

```
[user@controlnode ~]$ ansible servers --list-hosts -v
```

How to manage settings in configuration file?

- ✓ The Ansible configuration consist of several sections which each section consist of key-value pairs.
- ✓ for basic operations use following two sections:
- ✓ 1: `[defaults]` --> set defaults for ansible.
- ✓ 2: `[privilege_escalation]` --> configures how ansible performs privilege escalations on managed hosts.

`[defaults]`

```
inventory = ./inventory
```

```
remote_user = user
```

```
ask_pass = false
```

`[privilege_escalation]`

```
become = true
```

```
become_method = sudo
```

```
become_user = root
```

```
become_ask_pass = false
```

Explanation:

- ✓ **inventory:** Specifies the path to the inventory file.
- ✓ **remote_user:** The name of the user to log in as on the managed hosts. If not specified, the current user's name is used
- ✓ **ask_pass:** Whether or not to prompt for an SSH password. Can be false if using SSH public key authentication.
- ✓ **become:** Whether to automatically switch user on the managed host (typically to root) after connecting. This can also be specified by a play
- ✓ **become_method:** How to switch user (typically sudo, which is the default, but su is an option).
- ✓ **become_user:** The user to switch to on the managed host (typically root, which is the default)
- ✓ **become_ask_pass:** Whether to prompt for a password for your become_method. Defaults to false.
- ✓ By default, Ansible connects to managed hosts using the SSH protocol.
- ✓ The most important parameters that control how Ansible connects to the managed hosts are set in the [defaults] section.
- ✓ By default, Ansible attempts to connect to the managed host using the same user name as the local user running the Ansible commands.
- ✓ To specify a different remote user, set the remote_user parameter to that user name.

Non-SSH Connections:

- ✓ The protocol used by Ansible to connect to managed hosts is set by default to "smart", which determines the most efficient way to use SSH.
- ✓ This can be set to other values in a number of ways. For example, there is one exception to the rule that SSH is used by default. If you do not have localhost in your inventory, Ansible sets up an implicit localhost entry to allow you to run ad hoc commands and playbooks that target localhost. T
- ✓ This special inventory entry is not included in the "all" or "ungrouped" host groups.
- ✓ In addition, instead of using the "smart SSH" connection type, Ansible connects to it using the special "local" connection type by default.

```
[user@controlnode ~]$ ansible localhost --list-hosts
```

- ✓ The "local" connection type ignores the remote_user setting and runs commands directly on the local system. If privilege escalation is being used, it runs sudo from the user account that ran the Ansible command, not remote_user.
- ✓ This can lead to confusion if the two users have different sudo privileges.
- ✓ If you want to make sure that you connect to localhost using SSH like other managed hosts, one approach is to list it in your inventory. But this includes it in the all and ungrouped groups, which you may not want to do.
- ✓ Another approach is to change the protocol used to connect to localhost. The best way to do this is to set the "ansible_connection" host variable for localhost.
- ✓ To do this, in the directory from which you run Ansible commands, create a "host_vars" subdirectory. In that subdirectory, create a file named "localhost", containing the line "ansible_connection: smart". This ensures that the smart (SSH) connection protocol is used instead of local for localhost.
- ✓ You can use this the other way around as well. If you have 127.0.0.1 listed in your inventory, by default you will connect to it using smart. You can also create a "host_vars/127.0.0.1" file containing the line "ansible_connection: local" and it will use local instead

CONFIGURATION FILE COMMENTS:

- ✓ There are two comment characters allowed by Ansible configuration files: the hash or number sign (#) and the semicolon (;).
- ✓ The number sign at the start of a line comments out the entire line. It must not be on the same line with a directive.
- ✓ The semicolon character comments out everything to the right of it on the line. It can be on the same line as a directive, as long as that directive is to its left.

🔗 Follow my LinkedIn Profile: www.linkedin.com/in/muhammad-shaban-45577719a

🔗 Youtube Channel: <http://www.youtube.com/@engrm.shaban5099>